

Guía del Paquete C: Compliance Engine & Dashboard Monitor

Este documento describe la arquitectura, responsabilidades y guía de ejecución del **Paquete C (APP 3)**, desarrollado como el motor de evaluación de riesgos y auditoría (Compliance) del sistema OSINT.

🔗 Responsabilidad (Persona 3)

El **Paquete C** es el responsable de recibir los datos en crudo que han sido previamente extraídos y procesados por el equipo de OSINT, para evaluarlos contra listas restrictivas y de sanciones internacionales (OpenSanctions). Si se detecta un alto riesgo, el motor genera una alerta permanente para los analistas, que se visualiza en un Dashboard en tiempo real.

🏗️ Arquitectura y Tecnologías

El paquete se divide en dos componentes principales:

1. Backend (Compliance Engine):

- **Framework:** Java 17 puro + Spring Boot 3.x
- **Base de Datos Relacional:** PostgreSQL 15 (Almacenamiento permanente de alertas estructuradas).
- **Base de Datos Documental:** Elasticsearch 8 (Indexación de reportes OSINT completos para búsquedas rápidas).
- **Mensajería:** RabbitMQ (Consumo asíncrono de eventos y emisión de notificaciones de alerta).

2. Frontend (Dashboard Monitor):

- **Framework:** React.js + Vite.
- **Estilos:** TailwindCSS 4 + PostCSS.
- **Objetivo:** Single Page Application (SPA) para que los analistas revisen las alertas de riesgo (UI interactiva).

🔗 Integración por Eventos (RabbitMQ)

El Paquete C sigue una arquitectura orientada a eventos. Todo fluye a través del exchange `osint.exchange`.

Eventos que CONSUME (Input)

- **Routing Key:** `osint.completado`
- **Cola exclusiva:** `compliance.osint.completado.queue`
- **Descripción:** Escucha cuando la APP 1 termina un reporte. El JSON entrante debe contener toda la data extraída de las APIs gubernamentales y la URL del PDF.
- **Payload Esperado:**

```
{
  "requestId": "UUID",
  "targetId": "UUID",
  "pdfUrl": "s3://ruta/reporte.pdf",
  "status": "COMPLETED",
  "completedAt": 1715000000000,
  "data": {
    "fullName": "NOMBRE COMPLETO",
    "birthDate": "YYYY-MM-DD",
    "civilStatus": "Soltero/Casado"
  }
}
```

Eventos que EMITE (Output)

- **Routing Key:** `alerta.compliance`
- **Descripción:** Si el motor de OpenSanctions devuelve un riesgo mayor al 80% (score > 0.8), el sistema guarda la alerta y emite este evento para notificar al resto del ecosistema (por ejemplo, para que el Portal Ciudadano le notifique al usuario o para que se envíe un correo).
- **Payload Emitido:**

```
{
  "alertId": "UUID",
  "requestId": "UUID",
  "targetId": "UUID",
  "fullName": "NOMBRE COMPLETO",
  "riskLevel": "HIGH",
  "matchReason": "Razón del match",
  "timestamp": 1715000000000
}
```

API REST (Endpoints)

El Backend expone los siguientes endpoints para el Dashboard de React (Expuesto en `http://localhost:8081`):

- GET `/api/v1/compliance/alerts`: Retorna un arreglo JSON con todas las alertas de riesgo generadas, ordenadas descendientemente por fecha.
- GET `/api/v1/compliance/search?q={nombre}`: Permite hacer búsquedas directamente en Elasticsearch sobre los reportes almacenados.

Guía de Ejecución Local

Para levantar todo el entorno del Paquete C de forma local, sigue estos pasos:

1. Levantar la Infraestructura (Docker)

Asegúrate de que los contenedores correspondientes a la APP 3 estén levantados desde la raíz del proyecto:

```
docker-compose up -d app4-rabbitmq app3-postgres app3-elasticsearch
```

Nota: Postgres corre en el puerto local 5433 (compliance_user / compliance_password) y Elasticsearch en el 9200.

2. Levantar el Backend (Spring Boot)

Se eliminó la dependencia de Lombok para garantizar la compatibilidad con cualquier versión moderna del JDK.

```
cd app3-compliance/backend
./mvnw clean spring-boot:run
```

El servidor backend estará disponible en `http://localhost:8081`.

3. Levantar el Frontend (React Dashboard)

```
cd app3-compliance/frontend
npm install
npm run dev
```

El panel de analistas estará disponible en `http://localhost:5173`.

Pruebas Unitarias y de Integración (Mock)

Dado que la API real de OpenSanctions requiere de llaves de pago o es restrictiva, el Backend implementa un servicio Mock (`OpenSanctionsMockService`).

¿Cómo funciona el Mock?

Si publicas un evento en RabbitMQ (`osint.completado`) donde el campo `fullName` contenga el apellido **"PEREZ"** o la palabra **"SANCIONADO"**, el Mock automáticamente devolverá un *Score de Riesgo del 89% (0.89)*, disparando así una alerta roja de Compliance en el sistema, la cual podrás ver inmediatamente en tu Frontend. En cualquier otro caso, lo marcará como Riesgo Bajo (`score: 0.1`) y no generará alerta.